1 - Estilo Python: Patrones idiomáticos e Introducción a Pandas

En esta unidad, exploraremos las convenciones de estilo PEP 8 y los patrones idiomáticos de Python que permiten escribir código claro, eficiente y profesional. Aprenderás a aplicar comprehensions, unpacking y context managers para mejorar la legibilidad y modularidad de tus scripts, fundamentales para el desarrollo de proyectos de Data Science reproducibles y mantenibles.

¿Por qué es importante el estilo en Python?

Imagina que trabajas en un equipo de Data Science donde cada miembro escribe código con estilos diferentes: algunos usan espacios, otros tabulaciones; unos nombran variables con mayúsculas, otros con guiones bajos. El resultado es un código difícil de leer, mantener y compartir. Aquí es donde entra en juego PEP 8, la guía oficial de estilo para Python, que establece convenciones claras para escribir código legible y profesional.
Además, Python ofrece patrones idiomáticos que permiten escribir código más conciso y expresivo, como las comprehensions y el unpacking. En esta unidad, aprenderás a aplicar estas convenciones y patrones para que tu código no solo funcione, sino que sea elegante y fácil de entender, una habilidad clave para cualquier científico de datos que busca colaborar y construir proyectos reproducibles.

PEP 8: La guía de estilo para Python

PEP 8 es un documento que define las convenciones para escribir código Python legible y consistente. Algunas de sus recomendaciones clave incluyen:

- **Indentación: usar 4 espacios por nivel, evitar tabulaciones.**
- **Longitud de línea: máximo 79 caracteres para facilitar la lectura.**
- **Nombres: variables y funciones en snake_case, clases en CamelCase.**
- **Espacios: alrededor de operadores (=, +, -), después de comas.**
- **Importaciones: una por línea, ordenadas y agrupadas.**

```
# Correcto según PEP 8
def calcular_media(valores):
    suma = sum(valores)
    cantidad = len(valores)
    return suma / cantidad
```

Patrones idiomáticos en Python

Python tiene construcciones que permiten escribir código más compacto y legible:

Comprehensions

Permiten crear listas, diccionarios o conjuntos de forma concisa.

```
# Lista de cuadrados
cuadrados = [x**2 for x in range(10)]
```

Unpacking

Asignar múltiples variables en una sola línea o extraer valores de colecciones.

```
# Asignación múltiple
x, y, z = (1, 2, 3)
```

```
# Unpacking en funciones
def sumar(a, b, c):
    return a + b + c
```

```
valores = (4, 5, 6)
resultado = sumar(*valores)
```

Context Managers

Manejan recursos como archivos de forma segura y automática.

```
with open('datos.csv', 'r') as archivo:
    contenido = archivo.read()
```

Este patrón asegura que el archivo se cierre correctamente, incluso si ocurre un error.

Nota: Estos patrones no solo mejoran la legibilidad, sino que también previenen errores comunes y facilitan la colaboración en proyectos de Data Science.
En proyectos reales de Data Science, el código suele ser revisado y extendido por varios colaboradores. Aplicar PEP 8 y patrones idiomáticos facilita la comprensión y mantenimiento del código, especialmente en notebooks y pipelines reproducibles.

Por ejemplo, usar comprehensions para transformar datos es más eficiente y claro que usar bucles explícitos. El unpacking simplifica la manipulación de tuplas o listas de parámetros, y los context managers garantizan que los recursos se liberen correctamente, evitando errores difíciles de detectar.

Además, muchas herramientas de análisis y visualización en Python (como pandas y matplotlib) se benefician de un código limpio y modular, que es más fácil de depurar y optimizar.

Esta unidad te prepara para escribir código Python profesional, que es la base para construir funciones reutilizables y pipelines robustos en las siguientes unidades.

Introducción a Pandas

Pandas es una librería de Python diseñada para facilitar el manejo y análisis de datos. Construida sobre NumPy, Pandas extiende sus capacidades, proporcionando estructuras de datos y funciones avanzadas que permiten una manipulación más flexible y eficiente de datos tabulares y de series temporales.
Lanzada inicialmente en 2008, Pandas se ha convertido en una herramienta indispensable para científicos de datos, analistas y cualquier persona que trabaje con grandes volúmenes de datos. Forma parte del proyecto NUMFOCUS, que apoya el desarrollo de herramientas de código abierto para ciencia de datos.

Principales características de Pandas:

- **Series y DataFrames:** estructuras para trabajar con datos unidimensionales y bidimensionales.
- **Operaciones de manipulación de datos:** filtrado, transformación, agregación, etc.
- **Compatibilidad con múltiples formatos de datos:** CSV, Excel, SQL, JSON, entre otros.
- **Manejo de datos ausentes:** incluye funciones para detectar y tratar valores faltantes.

En resumen, Pandas es una librería poderosa y versátil que hace que el manejo de datos en Python sea más intuitivo y eficiente.

Series y DataFrames

En Pandas, las estructuras de datos fundamentales son:

- **Series:** Estructura unidimensional, similar a un array de NumPy, pero con índices explícitos. Todos los elementos deben ser del mismo tipo de dato. Todos los elementos deben ser del mismo tipo.
  Posee múltiples métodos útiles para análisis de datos.

- **DataFrames:** Estructura bidimensional (tabla), compuesta por varias Series. Tiene filas (con índice) y columnas (con nombre, pudiendo tener distintos tipos de datos). Permite selección, filtrado, agregación y combinación de datos.

---

2 - NumPy: arreglos, broadcasting y rendimiento

☆ ¿Qué es NumPy?
Es una librería optimizada escrita en C que permite manejar arreglos (arrays) multidimensionales y realizar operaciones matemáticas de forma rápida, segura y eficiente. Python puro es lento para cálculos numéricos, pero NumPy soluciona esto usando estructuras compactas y operaciones vectorizadas.

NumPy es una librería fundamental para el manejo de datos numéricos en Python, especialmente diseñada para realizar operaciones matemáticas y de álgebra lineal de manera eficiente.

Fue creada en 2005 como una evolución de las bibliotecas Numeric y Numarray, con el objetivo de optimizar el trabajo con grandes volúmenes de datos numéricos. Permite manipular y analizar datos de manera más rápida y eficiente.
NumPy introduce una estructura de datos llamada ndarray (N-dimensional array), que es un array multidimensional optimizado para operaciones numéricas. A diferencia de las listas de

Python, los elementos de un ndarray deben ser del mismo tipo de dato, lo que permite operaciones vectorizadas más rápidas y uso eficiente de la memoria.

Características clave de NumPy

- **Arreglos multidimensionales (ndarrays):** Soporta arrays de una, dos o más dimensiones, ideales para trabajar con matrices y tensores.
- **Operaciones matemáticas rápidas:** Las operaciones sobre ndarrays están altamente optimizadas.
- **Compatibilidad con otras librerías:** Sirve como base para bibliotecas como Pandas, SciPy y scikit-learn.

  En resumen, NumPy es una herramienta esencial para trabajar con datos numéricos en Python, mejorando la eficiencia en cálculos y análisis.

Tipos y atributos

En NumPy, los arrays (ndarrays) contienen elementos de un único tipo de dato, a diferencia de las listas de Python. Esta homogeneidad permite mayor velocidad y menor consumo de memoria.

Tipos de Datos

NumPy soporta una variedad de tipos de datos, entre los más comunes:

- **int:** Números enteros
- **float:** Números reales con decimales
- **bool:** Valores booleanos (True o False)
- **complex:** Números complejos (ej. 1.0 + 2.0j)
- **str:** Cadenas de texto
- **object:** Objetos arbitrarios
- **datetime y timedelta:** Fechas y diferencias temporales

El tipo de datos se puede definir usando el parámetro dtype al crear un array. Si no se especifica, NumPy lo infiere automáticamente.

Atributos de los ndarrays

Los arrays tienen atributos que permiten acceder a información relevante:

- **ndim:** Número de dimensiones.
- **shape:** Tupla que indica el tamaño en cada dimensión.
- **size:** Número total de elementos.
- **dtype:** Tipo de dato.
- **itemsize:** Tamaño en bytes de cada elemento.
- **nbytes:** Tamaño total en memoria.

Ej con uso de atributos

```
import numpy as np

# Crear un ndarray de ceros con 10 elementos

arr = np.zeros(10)

print(arr.ndim)       # Output: 1

print(arr.shape)      # Output: (10,)

print(arr.size)       # Output: 10

print(arr.dtype)      # Output: float64

print(arr.itemsize)   # Output: 8

print(arr.nbytes)     # Output: 80
```

Estos atributos son fundamentales para comprender la estructura del array y optimizar el uso de memoria y procesamiento.

Operaciones en NumPy

NumPy proporciona una serie de operaciones básicas que permiten manipular y transformar arrays de manera flexible y eficiente. Algunas de las operaciones más comunes son el reshape, la concatenación y el splitting. Estas operaciones son fundamentales para reestructurar, combinar y dividir datos almacenados en arrays, facilitando así el procesamiento de grandes volúmenes de datos.

Operaciones basicas:

Reshape: La operación reshape permite cambiar la forma de un array sin alterar sus datos. Es útil para reorganizar elementos y adaptarlos a estructuras multidimensionales.

```
import numpy as np

# Crear un array de 12 elementos

arr = np.arange(12)

# Cambiar la forma del array a una matriz de 3x4

arr_reshaped = arr.reshape((3, 4))

print(arr_reshaped)

Salida:

$$ 0  1  2  3$$$

4 5 6 74 5 6 7

$ 8  9 10 11$$$$
```

Concatenation: Une dos o más arrays (horizontal o verticalmente).

```
import numpy as np

# Crear dos arraysarr1 = np.array($1, 2, 3$)arr2 = np.array($4, 5, 6$)#

Concatenar horizontalmente (por defecto)arr_concatenated = np.concatenate((arr1,

arr2))

print(arr_concatenated)


Salida:

123456123456
```

Splitting: Divide un array en subarrays más pequeños.

```
import numpy as np
# Crear un array de 9 elementos
arr = np.arange(9)
# Dividir el array en 3 subarrays
arr_split = np.split(arr, 3)
print(arr_split)
```

```
Salida:

array(array(0, 1, 2),array(),array(3, 4, 5),array(),array(6, 7, 8))
```

Operaciones Vectorizadas (ufuncs)

Las operaciones vectorizadas (o *ufuncs* en NumPy) permiten aplicar una función a cada elemento de un array de manera eficiente, aprovechando el procesamiento paralelo de la CPU.

Ventajas

- **Eficiencia:** Ejecutan más rápido que los bucles tradicionales.
- **Simplicidad:** Requieren menos líneas de código.
- **Paralelismo:** Aprovechan múltiples núcleos del procesador.

Ejemplo

```
# Suma de un escalar a un array

import numpy as np

arr = np.array($1, 2, 3, 4$)

arr_sum = arr + 10

print(arr_sum)


Salida:

1112131411121314


Suma de dos arrays:

import numpy as np

arr1 = np.array($1, 2, 3$)arr2 = np.array($4, 5, 6$)arr_sum = arr1 + arr2

print(arr_sum)

Salida:

579579
```

```
Producto Matricial:

import numpy as np

mat1 = np.array($$1, 2$, $3, 4$$)

mat2 = np.array($$5, 6$, $7, 8$$)

mat_product = np.matmul(mat1, mat2)

print(mat_product)

Salida:

$$19 22$$$

$43 50$$$$
```

¿Cómo funciona internamente?

1. **Vectorización:** En lugar de usar loops en Python —que son lentos— NumPy ejecuta operaciones sobre arrays completos en una sola instrucción optimizada en C.

```
import numpy as np
a = np.array([1, 2, 3])b = a * 2   # Vectorización: no hay loop en Python
```

2. **Broadcasting:** NumPy realiza operaciones entre arrays de distintos tamaños aplicando reglas automáticas (ej. sumar 10 a cada elemento de un arreglo).

```
np.array([1, 2, 3]) + 10  # suma 10 a cada elemento
```

3. **Optimización a bajo nivel: Corre en C usando instrucciones vectoriales del CPU.**

Mucho del código de NumPy corre fuera de Python usando:

- C altamente optimizado
- instrucciones vectoriales del CPU
- manejo eficiente de memoria

Esto lo hace muchísimo más rápido.

¿Para qué se usa NumPy?

- Álgebra lineal
- Estadística
- Transformaciones numéricas
- Procesamiento de imágenes
- Machine Learning
- Manipulación eficiente de grandes volúmenes de datos

Todo lo que hagas en pandas, scikit-learn, TensorFlow o PyTorch está construido encima de NumPy.

---

3 - Pandas básico y Manipulación de Colecciones (Mutabilidad)

En Python, no todos los tipos de datos se comportan igual cuando se modifican. Esta diferencia es clave cuando trabajamos con funciones o estructuras grandes en Data Science.

Tipos mutables e inmutables

- Tipos inmutables: No pueden modificarse una vez creados (int, float, str, tuple). Cada operación crea un nuevo objeto en memoria.
- Tipos mutables: Pueden modificarse en el mismo lugar de memoria (list, dict, set, DataFrames). Esto los vuelve potencialmente peligrosos si no se manejan con cuidado.

Mutabilidad y el problema del Aliasing

En Python, las funciones reciben referencias a objetos, no copias automáticas.

```
def modificar(lista):
```

```
lista.append(100)
```

```
mi_lista = [1, 2, 3]
modificar(mi_lista)
print(mi_lista)  # Resultado: [1, 2, 3, 100]
```

La lista cambia fuera de la función porque list es mutable. Este comportamiento se llama Aliasing: dos variables diferentes apuntan al mismo objeto en la memoria.

¿Por qué usamos una copia profunda?

Para evitar efectos secundarios y alterar datasets originales:

- ❌**Copia superficial (Aliasing):** copia = lista_original
- ✅**Copia superficial independiente:** copia = lista_original.copy() (sirve para listas simples).
- ✅**Copia profunda (La más segura):**

```
import copy
copia = copy.deepcopy(lista_original)
```

Relación con el mundo real (Data Science): Este concepto es crítico en preprocesamiento de datos y Feature Engineering. Modificar datasets originales sin querer o reutilizar listas sucias genera resultados inconsistentes y errores difíciles de depurar. Entender la mutabilidad garantiza código seguro y predecible.

---

4 - Control de flujo avanzado y patrones

Explora estructuras condicionales complejas, técnicas eficientes para iteraciones, comprensión profunda de generadores y el uso práctico de herramientas como itertools para escribir código Python modular y mantenible en Data Science.

¿Alguna vez te has encontrado escribiendo código con múltiples condicionales anidados o iteraciones que parecen poco claras y difíciles de mantener? En el análisis de datos, la claridad y eficiencia del código son esenciales para manejar grandes volúmenes de información y construir pipelines reproducibles. En esta unidad, profundizaremos en estructuras de control de flujo avanzadas, exploraremos cómo crear funciones reutilizables y descubriremos herramientas poderosas como los generadores y el módulo itertools que te ayudarán a escribir código Python más idiomático y eficiente. Al finalizar, podrás seleccionar y aplicar patrones adecuados para controlar el flujo de tus programas y optimizar iteraciones, habilidades clave para cualquier científico de datos que busca calidad y rendimiento en sus proyectos.

Control de flujo avanzado y funciones reutilizables

Condicionales complejas

Las estructuras condicionales permiten tomar decisiones en el código. Más allá de simples if y else, Python soporta condicionales anidados y expresiones condicionales (operador ternario) que facilitan la escritura clara y concisa.

```
# Condicional anidado
if edad >= 18:
    if tiene_licencia:
            print("Puede conducir")
    else:
            print("No tiene licencia")
else:
        print("Menor de edad")
```

```
# Expresión condicional
estado = "Adulto" if edad >= 18 else "Menor"
```

> *Nota: Evita anidar demasiadas condicionales para mantener la legibilidad.*

Técnicas avanzadas de iteración

Python ofrece varias formas eficientes de iterar:

- **Bucles for con enumerate() para obtener índice y valor al mismo tiempo.**
- **Comprensiones de listas para creación concisa.**
- **Funciones generadoras para manejar secuencias inmensas sin saturar la memoria.**

Generadores en Python (yield) Un generador devuelve un iterador que produce valores bajo demanda. Son útiles para manejar grandes conjuntos de datos o streams optimizando la memoria RAM.

```
# Enumerate
for i, valor in enumerate(lista):
    print(f"Índice {i}: {valor}")

# Comprensión de listas
cuadrados = [x**2 for x in range(10)]

# Generador simple
def cuenta_regresiva(n):
    while n > 0:
        yield n
        n -= 1
```

Herramientas itertools

El módulo estándar provee funciones para crear combinaciones complejas de forma eficiente:

- **count():** Contador infinito.

- **cycle():** Ciclo infinito sobre una secuencia.
- **chain():** Encadena múltiples iterables.
- **islice():** Porciona iterables.

```
import itertools

# Ciclo infinito sobre una lista
colores = ['rojo', 'verde', 'azul']
for color in itertools.cycle(colores):
    print(color)
    break  # para evitar ciclo infinito en ejemplo
```

Patrones para funciones reutilizables

Crear funciones claras y modulares mejora la mantenibilidad:

- **Funciones puras:** Sin efectos secundarios (no modifican el original). Las mismas entradas siempre producen las mismas salidas.
- **Funciones puras:** sin efectos secundarios, mismas entradas producen mismas salidas
- Funciones con argumentos por defecto y variables para flexibilidad
- **Closures:** funciones que recuerdan el estado del entorno donde fueron creadas

```
def multiplicador(factor):
def multiplicar(n):
return n * factor
return multiplicar
```

```
doblar = multiplicador(2)
print(doblar(5))  # Resultado: 10
```

En proyectos de Data Science, el manejo eficiente del control de flujo y la creación de funciones reutilizables son fundamentales para construir pipelines reproducibles y escalables. Por ejemplo, al procesar grandes datasets, usar generadores evita cargar toda la información en memoria, mejorando el rendimiento. Herramientas como itertools permiten combinar y manipular iteradores de forma elegante y eficiente, facilitando tareas comunes como la combinación de múltiples fuentes de datos o la creación de ventanas deslizantes para análisis temporal.

Además, adoptar patrones de diseño simples, como funciones puras y closures, ayuda a mantener el código limpio y modular, lo que es crucial cuando se trabaja en equipo o se mantiene código a largo plazo. Esta unidad te prepara para aplicar estas técnicas en tus notebooks y scripts, mejorando la calidad y eficiencia de tus análisis.

5 - Visualizaciones: Buenas prácticas y Seaborn

Introducción a las buenas prácticas para trabajar con notebooks en JupyterLab y Google Colab, enfocándose en la organización, documentación, manejo de entornos, uso de magics y exportación para asegurar la reproducibilidad en proyectos de Data Science.

Introducción a la Visualización de Datos

La visualización de datos es un componente esencial en la ciencia de datos, ya que permite comunicar de manera efectiva los resultados del análisis. Sin embargo, es importante recordar que la visualización de datos conlleva una gran responsabilidad. Como lo sugiere el Principio de Peter Parker: "Con un gran poder viene una gran responsabilidad". Este principio subraya la importancia de ser cuidadosos y éticos en la presentación de datos, ya que estos pueden influir en la toma de decisiones.

Importante

La visualización de datos no es simplemente elegir un gráfico al azar, sino que requiere una preparación detallada para asegurar que la información se presenta de manera clara, precisa y fiel a

los datos reales. Un gráfico debe ser capaz de comunicar mucha información con pocos elementos, garantizando que sea comprensible y que represente adecuadamente la realidad que los datos buscan describir.

Elementos de un Buen Gráfico

Para que un gráfico sea efectivo, debe incluir ciertos elementos esenciales que aseguren la claridad y precisión en la comunicación de los datos:

1. **Título:** Debe ser conciso y representativo, indicando claramente la información que se pretende mostrar.
2. **Etiquetas y Unidades**: Es crucial que las variables involucradas en el gráfico estén claramente etiquetadas, incluyendo el nombre de cada variable y las unidades de medición correspondientes. Esto permite que el lector entienda exactamente qué se está representando.
3. **Leyendas y Categorías:** Si el gráfico incluye diferentes categorías o variables, es necesario que estas estén bien diferenciadas y claramente representadas. Esto incluye una leyenda que explique los colores, formas u otros elementos distintivos que se utilizan para representar distintas categorías o series de datos.

   Un gráfico que cumpla con estos elementos básicos facilitará la comprensión y permitirá que los datos sean comunicados de manera efectiva.

Diferencias entre Seaborn y Matplotlib

Característica	Matplotlib	Seaborn
Funcionalidad	Ofrece gráficos básicos	Especializado en gráficos estadísticos más complejos
Sintaxis	Sintaxis larga y más compleja (e.g., matplotlib.pyplot.bar(x_axis, y_axis))	Sintaxis más simple y legible (e.g., seaborn.barplot(x_axis, y_axis))
Múltiples Figuras	Permite manejar múltiples figuras simultáneamente	Puede tener problemas de memoria en gráficos complejos
Flexibilidad	Altamente customizable y robusto	Menos flexible, pero evita superposiciones innecesarias
Manejo de Datos	Trabaja eficientemente con arrays y DataFrames, tratando figuras y ejes como objetos	Más orientado a manejar datasets como unidades simples
Casos de Uso	Útil para una amplia variedad de gráficos, especialmente aquellos derivados de Numpy y Pandas	Esencialmente una extensión de Matplotlib con capacidades avanzadas para datos estadísticos (Copia de Clase 9 - Visu...)

Conclusión

Notamos así cómo Seaborn se enfoca en simplificar la creación de gráficos estadísticos y trabajar directamente con estructuras de datos, mientras que Matplotlib ofrece una mayor flexibilidad y robustez, pero a costa de una sintaxis más compleja.

Tipos de Gráficos

Gráficos de Barras

Los gráficos de barras son una herramienta fundamental en la visualización de datos, especialmente cuando se trata de representar datos categóricos. Este tipo de gráfico utiliza barras rectangulares cuya longitud es proporcional a los valores que representan, facilitando la comparación entre diferentes categorías.

Las barras se pueden organizar tanto horizontal como verticalmente. En un gráfico de barras vertical, el eje horizontal contiene las categorías y el eje vertical muestra la magnitud de cada categoría. En un gráfico de barras horizontal, esta disposición se invierte, lo que puede ser útil cuando las etiquetas de las categorías son largas o cuando se tienen muchas categorías que dificultan la lectura.

Es importante destacar que los gráficos de barras no son adecuados para representar datos que varían con el tiempo. En estos casos, las relaciones y tendencias son más difíciles de visualizar, y es preferible utilizar gráficos de líneas, que son más efectivos para mostrar la evolución de los datos a lo largo del tiempo.

Recomendaciones para el uso de gráficos de barras:

Utilízalos cuando necesites comparar cantidades entre diferentes grupos o categorías.

Evita usarlos para representar series de tiempo o cualquier dato que tenga un orden inherente en el tiempo; para estos casos, los gráficos de líneas son más apropiados.

Asegúrate de que las barras estén claramente etiquetadas y que las diferencias en las alturas de las barras sean fácilmente distinguibles para evitar malinterpretaciones.

Diagramas de Dispersión

Los diagramas de dispersión, o *scatterplots*, son una herramienta visual clave para analizar y explorar la relación entre dos variables. En un diagrama de dispersión, cada punto en el gráfico representa una observación con dos valores, uno para cada una de las variables en cuestión. La posición del punto en el eje horizontal corresponde al valor de una variable, mientras que la posición en el eje vertical corresponde al valor de la otra variable.

Este tipo de gráfico es especialmente útil para identificar patrones, tendencias y posibles correlaciones entre las dos variables. Por ejemplo, si los puntos en el gráfico tienden a alinearse en una dirección diagonal (ascendente o descendente), esto puede indicar una correlación entre las variables.

Gráficos de Torta

Los **gráficos de torta** son una herramienta de visualización de datos que se utiliza para mostrar la proporción de diferentes categorías en un conjunto de datos. Cada categoría se representa como un sector de un círculo (la "torta"), y la magnitud de cada sector es proporcional al porcentaje o parte que la categoría representa del total.

Estos gráficos son particularmente útiles cuando se desea resaltar la relación de partes con el todo, como por ejemplo, la distribución porcentual de ventas entre diferentes productos.

Características y Uso

- **Proporciones y Comparaciones**: Los gráficos de torta son ideales para comparar proporciones dentro de un conjunto limitado de categorías. Son útiles para responder preguntas como "¿Qué parte del total representa cada categoría?".
- **Limitación de Categorías**: Funciona mejor con un número reducido de categorías. Si hay demasiadas categorías, el gráfico se vuelve difícil de interpretar, ya que los sectores se vuelven demasiado pequeños.
- **Etiquetas y Colores**: Es importante etiquetar claramente cada sector y usar colores distintos para ayudar a distinguir visualmente las diferentes categorías.

Recomendaciones para el uso de gráficos de torta:

- Utilízalos cuando las categorías representadas suman un todo significativo y cuando es importante mostrar cómo se divide ese todo.

- Evita usarlos cuando hay muchas categorías o cuando las diferencias en tamaño entre los sectores son mínimas, ya que es difícil visualizar estas diferencias con precisión.
- Considera alternativas como los gráficos de barras apiladas si necesitas comparar varias proporciones de diferentes conjuntos de datos.

Gráficos de Series Temporales

- Los gráficos de series temporales son herramientas clave en la visualización de datos cuando se trata de representar cómo cambia una variable a lo largo del tiempo. Este tipo de gráfico utiliza líneas que conectan puntos de datos en un gráfico cartesiano, donde el eje horizontal representa el tiempo y el eje vertical representa los valores de la variable de interés.
- Características y Uso
- Tendencias y Patrones: Son ideales para identificar tendencias, patrones estacionales, y ciclos en los datos a lo largo del tiempo. Este tipo de gráfico responde bien a preguntas como "¿Cómo ha cambiado esta variable a lo largo del tiempo?".
- Comparación de Series: Se pueden comparar múltiples series temporales en el mismo gráfico para analizar cómo diferentes variables evolucionan en paralelo o cómo interactúan entre sí.
- Zoom y Detalle: En gráficos de series temporales, a menudo es útil poder hacer zoom para ver detalles específicos o panorámicas para ver tendencias generales.

Recomendaciones para el uso de gráficos de series temporales:

- Utilízalos cuando quieras mostrar cómo una variable cambia con el tiempo, especialmente para detectar tendencias o estacionalidad.
- Asegúrate de etiquetar claramente los ejes, especialmente el eje temporal, y usa colores distintos si representas múltiples series para evitar confusiones.
- Evita usarlos para datos categóricos no ordenados temporalmente, ya que puede llevar a una mala interpretación de la relación entre los puntos de datos.

Herramientas avanzadas de visualización

Introducción a Seaborn

Seaborn es una poderosa librería de visualización en Python que se construye sobre Matplotlib y se integra de manera fluida con Pandas, lo que la convierte en una herramienta preferida para la visualización de datos estadísticos. Seaborn está diseñada para hacer que la creación de gráficos sea más intuitiva y estéticamente agradable, ofreciendo un conjunto de funciones que simplifican la elaboración de visualizaciones complejas con poco código.

Uno de los principales atractivos de Seaborn es su capacidad para trabajar directamente con DataFrames de Pandas, lo que facilita la generación de gráficos a partir de conjuntos de datos estructurados. Esto significa que no es necesario extraer columnas individuales o manipular los datos antes de graficarlos; Seaborn puede manejar los DataFrames de manera eficiente y permite realizar análisis exploratorios rápidamente.

Además, Seaborn viene con una variedad de temas predefinidos y estilos gráficos que mejoran la presentación visual de los gráficos. Estos estilos están diseñados para proporcionar gráficos con una apariencia profesional sin necesidad de personalización adicional, aunque la librería también ofrece la flexibilidad de ajustar estos estilos según sea necesario.

En resumen, Seaborn es una herramienta esencial para cualquier científico de datos que desee realizar análisis visuales rápidos y eficaces, especialmente cuando se trabaja con datos estadísticos y series temporales en Python.

Comandos Básicos de Seaborn

Seaborn proporciona una serie de comandos básicos que facilitan la creación y personalización de gráficos. Uno de los comandos más importantes es sns.set(), que permite establecer los estilos gráficos predeterminados para todos los gráficos que se creen a partir de ese momento. Este comando es especialmente útil para garantizar una coherencia visual en las presentaciones o reportes, permitiendo ajustar aspectos como la paleta de colores, la visibilidad de la cuadrícula, y otros detalles estéticos con una sola línea de código.

Otra característica clave de Seaborn es la distinción entre funciones Axes-level y Figure-level:

Axes-level: Estas funciones, como sns.barplot() o sns.scatterplot(), operan directamente sobre un objeto de ejes de Matplotlib (matplotlib.pyplot.Axes). Son útiles cuando se desea un control preciso sobre un solo gráfico y permiten una mayor integración con la interfaz orientada a objetos de Matplotlib. Estas funciones no modifican más allá del gráfico en el que se dibujan.

Figure-level: Estas funciones, como sns.relplot() o sns.catplot(), manejan toda la figura, lo que incluye la creación de subgráficos y la organización de múltiples gráficos en una sola figura. Las funciones de Figure-level son más convenientes cuando se necesita construir gráficos complejos que involucren varias capas o subplots, ya que simplifican la gestión de leyendas, títulos y layouts.

Uso de Subplots y FacetGrid

- La creación de subgráficos, o subplots, es una técnica fundamental en la visualización de datos, permitiendo la comparación de múltiples gráficos en una sola figura. Seaborn y Matplotlib ofrecen varias herramientas para crear y personalizar subplots de manera eficiente.

- El comando plt.subplots() de Matplotlib es el punto de partida para crear subgráficos. Este comando permite especificar el número de filas (nrows) y columnas (ncols) que tendrá la grilla de subgráficos. El resultado es un array de objetos Axes, sobre los cuales se pueden dibujar gráficos individuales.

```
import matplotlib.pyplot as pltimport seaborn as sns
# Crear una figura con 2 filas y 2 columnas de subplots
fig, axs = plt.subplots(nrows=2, ncols=2, figsize=(10, 8))
# Graficar en cada subplotsns.histplot(data=df, x='variable1', ax=axs$0,
0$)sns.boxplot(data=df, x='variable2', ax=axs$0, 1$)sns.scatterplot(data=df,
x='variable3', y='variable4', ax=axs$1, 0$)sns.lineplot(data=df, x='variable5',
y='variable6', ax=axs$1, 1$)plt.tight_layout()plt.show()
```

Personalización

- Seaborn no solo facilita la creación de gráficos estadísticos, sino que también ofrece una amplia gama de opciones para personalizar esos gráficos, permitiéndote adaptarlos a las necesidades específicas de tus análisis y presentaciones. La personalización de gráficos en Seaborn incluye modificar estilos, colores, etiquetas y más, para asegurar que las visualizaciones sean no solo informativas, sino también estéticamente agradables y coherentes con el estilo deseado.

Estilos

- Seaborn proporciona varios estilos predefinidos que se pueden aplicar fácilmente a todos los gráficos. Los estilos más comunes incluyen "darkgrid", "whitegrid", "dark", "white" y "ticks". Estos estilos afectan la apariencia general del gráfico, incluyendo el fondo, la cuadrícula y las marcas de los ejes.

```
import seaborn as sns
# Aplicar un estilo de Seaborn
sns.set_style("whitegrid")
```

- El comando sns.set_style() permite aplicar un estilo específico a todos los gráficos creados después de su invocación, lo que es útil para mantener una coherencia visual en los reportes.

Colores

- Además de los estilos, Seaborn también ofrece la posibilidad de personalizar las paletas de colores utilizadas en los gráficos. Puedes elegir entre paletas predefinidas como "deep", "muted", "bright", "pastel", "dark", y "colorblind", o crear tu propia paleta personalizada.

```
# Establecer una paleta de colores
sns.set_palette("muted")
```

- También puedes personalizar colores específicos en elementos gráficos como líneas, barras o puntos, usando argumentos dentro de las funciones de Seaborn.

Etiquetas

- Personalizar las etiquetas de los ejes y el título del gráfico es esencial para asegurar que los gráficos sean interpretados correctamente. Seaborn facilita la adición y modificación de etiquetas y títulos directamente dentro de sus funciones, o utilizando funciones de Matplotlib.

```
# Crear un gráfico con etiquetas personalizadas
ax = sns.scatterplot(x="total_bill", y="tip", data=tips)ax.set(
    title="Relación entre Total de la Cuenta y Propina",
    xlabel="Total de la Cuenta ($)",
    ylabel="Propina ($)"
)
```

Ejemplos de Personalización

- Aquí tienes un ejemplo completo que combina varios aspectos de la personalización en Seaborn:

```
import seaborn as sns

# Establecer el estilo y la paleta

sns.set_style("darkgrid")

sns.set_palette("bright")

# Crear un gráfico de dispersión con etiquetas personalizadas

ax = sns.scatterplot(x="total_bill", y="tip", hue="day", data=tips)

ax.set(

    title="Relación entre Total de la Cuenta y Propina",

    xlabel="Total de la Cuenta ($)",

    ylabel="Propina ($)"

)

# Personalizar el tamaño de la leyenda y la posición del título

ax.legend(title="Día de la Semana", fontsize=12)

ax.title.set_position($0.5, 1.05$)

sns.despine()  # Eliminar bordes superiores y derechos del gráfico
```

- Este ejemplo demuestra cómo personalizar diferentes aspectos de un gráfico de dispersión, incluyendo el estilo, la paleta de colores, las etiquetas y la disposición de la leyenda y el título.

---

6 - Visualización con matplotlib: gráficos estáticos

En esta unidad aprenderás a crear gráficos estáticos informativos utilizando matplotlib, una biblioteca fundamental para la visualización de datos en Python. Practicarás la creación de gráficos de línea, barras, histogramas y dispersión, además de personalizar leyendas, ejes y subplots para comunicar resultados de manera clara y profesional.

¿Qué es Matplotlib?

Es una librería que funciona como el “equivalente en Python de MATLAB para gráficos”.

Proporciona una API flexible para generar visualizaciones en 2D y 3D, y es la base sobre la que se construyen librerías más modernas como Seaborn o Pandas.plot().

¿Cómo funciona?

Matplotlib se organiza principalmente en dos niveles:

1. Pyplot

Es un módulo que funciona de manera parecida a una calculadora gráfica:

Cada comando modifica o crea una figura.

Ejemplo básico:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4], [10, 20, 25,
30])plt.xlabel("X")plt.ylabel("Y")plt.title("Gráfico de línea simple")plt.show()
```

2. Sistema de figuras y ejes (Figure / Axes)

Permite un control más profesional y detallado:

Figure → el lienzo general

Axes → cada gráfico dentro del lienzo

Axis → los ejes X e Y

Ejemplo:

```
fig, ax = plt.subplots()ax.plot([1,2,3], [4,5,6])ax.set_title("Mi gráfico")plt.show()
```

¿Por qué es tan usado?

✓ Muy flexible y customizable

✓ Compatible con NumPy, Pandas y Scikit-learn

✓ Funciona en notebooks, scripts y aplicaciones

✓ Permite desde gráficos simples hasta dashboards complejos

En esta unidad, exploraremos cómo utilizar matplotlib para crear gráficos estáticos que permitan comunicar hallazgos de datos de forma clara y efectiva. Aprenderás a generar diferentes tipos de gráficos básicos y a personalizarlos para mejorar su legibilidad y presentación.

Conceptos clave

- matplotlib: biblioteca de Python para crear gráficos estáticos, animados e interactivos.
- Gráficos básicos: lineales, de barras, histogramas y de dispersión.
- Personalización: leyendas, etiquetas de ejes, títulos, colores y subplots.

Ejemplo básico

python

```
import matplotlib.pyplot as plt
# Datos de ejemplox = [1, 2, 3, 4, 5]y = [10, 15, 13, 17, 20]
# Gráfico de líneaplt.plot(x, y, label='Ventas')plt.xlabel('Día')plt.ylabel('Cantidad')plt.title('Ventas diarias')plt.legend()plt.show()
```

Marco teórico — Visualización con Matplotlib en contextos industriales

---

1. Visualización de datos en la Industria 4.0

En entornos industriales modernos (Industria 4.0), los sistemas generan datos de forma continua a partir de:

- sensores de temperatura y vibración
- líneas de producción automatizadas
- consumo energético
- costos operativos
- indicadores de eficiencia

Estos datos, por sí solos, no generan valor.
El valor aparece cuando pueden interpretarse rápidamente para apoyar la toma de decisiones operativas y estratégicas.
Aquí es donde la visualización de datos cumple un rol fundamental.
Un gráfico bien construido permite:

- detectar anomalías
- comparar rendimientos
- identificar tendencias
- anticipar fallas
- comunicar información técnica a perfiles no técnicos

Por este motivo, antes de construir dashboards complejos, es esencial dominar la visualización estática, base de todo sistema analítico.

---

2. ¿Por qué usar Matplotlib?

Matplotlib es la librería fundamental de visualización en Python y cumple un rol estructural dentro del ecosistema de análisis de datos.

Características clave:

- Es la base de librerías como Seaborn y pandas.plot()
- Permite control total sobre cada elemento del gráfico

- Funciona en notebooks, scripts y aplicaciones
- Es ideal para análisis exploratorio, reportes técnicos y documentación

En entornos reales, Matplotlib se utiliza para:

- informes operativos
- reportes de performance
- análisis exploratorio previo a modelos
- validación visual de KPIs
- generación de gráficos exportables (PNG, PDF)

Por esta razón, dominar Matplotlib es un requisito profesional para cualquier Data Scientist o Data Analyst.

---

3. Estructura conceptual de Matplotlib

Matplotlib trabaja con una jerarquía clara:

⬙ Figure

Es el lienzo completo donde se dibuja uno o varios gráficos.

⬙ Axes

Representa cada gráfico individual dentro de la figura.

⬙ Axis

Corresponde a los ejes X e Y.

En la práctica profesional se utiliza principalmente:

```
fig, ax = plt.subplots()
```

Este enfoque (Figure / Axes):

- brinda mayor control
- permite crear subplots

- facilita dashboards
- evita errores visuales
- es el estándar en proyectos reales

Por eso, en esta práctica se prioriza este enfoque y no únicamente plt.plot().

---

4. Contexto del dataset utilizado

El dataset simula una planta industrial con:

Variables categóricas

- planta
- línea de producción
- producto
- turno
- operador

Variables numéricas

- temperatura
- vibración
- unidades producidas
- consumo energético
- costo unitario
- revenue

Además, la variable temporal permite analizar:

- evolución diaria
- tendencias

- comportamiento acumulado

Este tipo de estructura es muy similar a datos reales provenientes de sistemas MES o ERP industriales.

---

5. Lógica general de la práctica

La práctica sigue un **flujo analítico real**, similar al que se utiliza en la industria:

1. Comprender el negocio
2. Identificar KPIs relevantes
3. Elegir el tipo de gráfico adecuado
4. Construir visualizaciones claras
5. Interpretar resultados

Cada gráfico responde a una **pregunta de negocio concreta**.

---

6. Justificación de cada visualización

📈 Gráfico 1 — Serie temporal

Pregunta de negocio:

¿Cómo evoluciona la producción a lo largo del tiempo?

Se utiliza un gráfico de línea porque:

- permite observar tendencias
- facilita detectar picos o caídas
- es ideal para variables temporales

En este caso se analizan las **unidades producidas por día**, uno de los KPIs más importantes en producción.

Conceptos trabajados:

- series temporales
- formato de fechas

- rotación de etiquetas
- lectura cronológica

📊 Gráfico 2 — Gráfico de barras

Pregunta de negocio:

¿Qué línea de producción rinde mejor en promedio?

El gráfico de barras es ideal para:

- comparar categorías
- visualizar diferencias entre grupos
- apoyar decisiones operativas

Aquí se comparan las **líneas de producción**, una decisión típica en gestión industrial.

Conceptos trabajados:

- agregación
- comparación categórica
- interpretación de promedios

📉 Gráfico 3 — Histograma

Pregunta de negocio:

¿Cómo se distribuye la vibración de las máquinas?

El histograma permite:

- analizar la distribución de una variable
- detectar valores atípicos
- evaluar dispersión

Esto es clave en mantenimiento predictivo.

Conceptos trabajados:

- frecuencia
- bins
- forma de la distribución
- variabilidad

🟠 Gráfico 4 — Scatter plot

Pregunta de negocio:

¿Existe relación entre producción y consumo energético?

El scatter permite:

- analizar correlaciones
- detectar comportamientos no lineales
- observar patrones

Este tipo de análisis es muy común en eficiencia energética industrial.

Conceptos trabajados:

- relación entre variables
- dispersión
- interpretación visual de dependencia

🎁 Gráfico 5 — Boxplot

Pregunta de negocio:

¿Cómo varían los costos unitarios según el producto?

El boxplot permite:

- comparar distribuciones
- detectar outliers

- observar mediana y dispersión

Es ampliamente usado en control de costos y calidad.

Conceptos trabajados:

- mediana
- cuartiles
- valores extremos
- comparación entre grupos

🔀 Gráfico 6 — Doble eje (twin axis)

Pregunta de negocio:

¿Cómo se relacionan producción y revenue en el tiempo?

Este gráfico permite mostrar:

- dos métricas distintas
- con escalas diferentes
- en el mismo eje temporal

Muy utilizado en dashboards ejecutivos.

Conceptos trabajados:

- ejes secundarios
- combinación de gráficos
- lectura comparativa

□ Subplots — Panel resumen

Objetivo:

Construir una vista general del sistema productivo.

Los subplots permiten:

- sintetizar información
- comparar KPIs rápidamente
- simular dashboards estáticos

Es el paso previo natural antes de usar herramientas como Power BI, Tableau o Streamlit.

Conceptos trabajados:

- layouts
- organización visual
- lectura ejecutiva

7. Relación con el mundo profesional

Este flujo es el mismo que se utiliza en:

- análisis exploratorio (EDA)
- reportes ejecutivos
- validación de KPIs
- preparación de dashboards
- análisis previo a machine learning

Dominar estas visualizaciones es un paso obligatorio antes de:

- automatizar reportes
- construir dashboards interactivos
- entrenar modelos predictivos
- comunicar resultados a stakeholders

---

7 - Funciones: diseño, argumentos y closures

Ejercicio para reforzar el diseño de funciones claras y reutilizables en Python, manejo de argumentos variables (*args, **kwargs) y comprensión de closures y scopes.

Contexto

En análisis de datos y pipelines reproducibles, es común diseñar funciones que sean reutilizables y configurables mediante argumentos variables. Además, el uso de closures permite encapsular comportamientos y estados, facilitando la modularidad y mantenibilidad del código.

Marco teórico

Funciones, argumentos variables y *closures* en Python

En esta práctica trabajaremos con funciones como objetos de primera clase, el uso de argumentos variables (*args, **kwargs) y el concepto de closure para mantener estado interno entre llamadas. Estos conceptos son fundamentales en Python moderno y aparecen frecuentemente en contextos de data science, pipelines de procesamiento, APIs y diseño de software modular.***

La práctica se apoya en un dataset de negocio simulado, pero el foco no está en el análisis de datos sino en cómo diseñar funciones flexibles y reutilizables que operan sobre datos numéricos reales.

1-Funciones como objetos en Python

En Python, las funciones son objetos de primera clase, lo que significa que:

- Pueden asignarse a variables
- Pueden almacenarse en estructuras como listas o diccionarios
- Pueden pasarse como argumentos a otras funciones
- Pueden devolverse como resultado de una función

En esta práctica, esto se ve reflejado en dos aspectos clave:

- Las **operaciones matemáticas** (`sumar`, `multiplicar`, `potencia`) se definen como funciones independientes.
- Dichas funciones se almacenan en un diccionario (`OPERACIONES_DISPONIBLES`) y luego se pasan dinámicamente a otra función.

Esto simula escenarios reales donde:

- Las transformaciones no están “hardcodeadas”

- El comportamiento del programa depende de configuraciones externas (listas de operaciones)

2-Argumentos variables: *args y **kwargs***

Las funciones definidas en la práctica aceptan argumentos variables, lo cual permite mayor flexibilidad:

- `*args`: captura argumentos posicionales adicionales
- `**kwargs`: captura argumentos con nombre adicionales***

¿Por qué usamos **kwargs?**

En esta práctica, todas las operaciones reciben **kwargs aunque no usen todos los parámetros. Esto permite que:**

- Las operaciones compartan una **interfaz común**
- La función principal (`calculadora`) pueda pasar los mismos argumentos a todas las operaciones sin preocuparse por cuáles usa cada una

Esto es muy común en:

- Pipelines de machine learning
- Frameworks de procesamiento de datos
- Callbacks y funciones genéricas

Ejemplo conceptual:

op(x, **kwargs)

Cada operación decide internamente qué argumentos usar.

3-Closures: funciones con estado interno

Un closure es una función que:

- Se define dentro de otra función
- Recuerda y puede modificar variables del contexto donde fue creada, incluso después de que la función externa terminó de ejecutarse

En esta práctica, `crear_calculadora` devuelve una función interna llamada `calculadora` que:

- Tiene acceso a la variable `acumulador`

- Puede modificarla usando la palabra clave `nonlocal`
- Mantiene su valor entre múltiples llamadas

¿Por qué no usar una variable global?

Usar un closure permite:

- Evitar efectos colaterales globales
- Encapsular el estado
- Crear múltiples calculadoras independientes, cada una con su propio acumulador

Este patrón es común en:

- Contadores
- Métricas acumuladas
- Sistemas de logging
- Procesamiento por lotes

4-Diseño de la función crear_calculadora

La función crear_calculadora cumple varios objetivos de diseño importantes:

✓ Conversión dinámica de operaciones

La lista operaciones puede contener:

- Strings (nombres de operaciones)
- Funciones directamente

Esto obliga a:

- Validar tipos
- Traducir nombres a funciones
- Manejar errores de forma explícita

✓ Aplicación secuencial

Cada número pasa por una cadena de transformaciones, simulando un pipeline:

```
x → operación 1 → operación 2 → ... → resultado final
```

✓ Estado persistente

El acumulador se actualiza cada vez que se procesa un número y persiste entre llamadas a la función retornada.

Conclusión

Esta práctica no busca complejidad matemática, sino calidad de diseño y comprensión profunda de funciones en Python, sentando bases sólidas para temas más avanzados como:

- Decoradores
- Pipelines de datos
- Callbacks
- Arquitecturas funcionales en data science

---

8 - Testing ligero y depuración en funciones

Escribir la función es el 50% del trabajo. El otro 50% es asegurar que sea robusta y se comporte de forma predecible ante datos inesperados. Un error que pase desapercibido en la limpieza de datos invalidará todo el modelo posterior.

Introducción

Cuando desarrollamos funciones reutilizables, es fundamental asegurarnos de que funcionan correctamente. Para ello, usamos técnicas de depuración y pruebas unitarias ligeras que nos permiten detectar errores de forma rápida y mantener la calidad del código.

En esta unidad, aprenderás a:

- Usar print y depuradores para entender el flujo y estado de tus funciones.
- Escribir pruebas simples con assert para validar resultados esperados.

Técnicas básicas de depuración

- **Print debugging:** Insertar `print()` en puntos clave para inspeccionar valores.
- **Depuradores:** Herramientas que permiten pausar la ejecución y examinar variables.

Pruebas unitarias simples

Usar assert para comprobar que una función devuelve el resultado esperado:

python
```
assert mi_funcion(entrada) == salida_esperada
```

Esto ayuda a detectar errores automáticamente.

Ejemplo práctico

Supongamos que tenemos una función que calcula el máximo común divisor (MCD) de dos números.

python
```
def mcd(a, b):
        while b != 0:
        a, b = b,
        a % b
return a
```

Podemos probarla con asserts:

python
```
assert mcd(12, 8) == 4
assert mcd(100, 25) == 25
assert mcd(7, 3) == 1
```

Si alguna falla, Python lanzará un AssertionError.

9 - Tipos, colecciones y mutabilidad en profundidad

Ejercicio para comprender y aplicar conceptos de tipos de datos básicos y colecciones en Python, enfocándose en mutabilidad, aliasing y efectos secundarios en funciones.

Introducción

Cuando desarrollamos funciones reutilizables, es fundamental asegurarnos de que funcionan correctamente. Para ello, usamos técnicas de depuración y pruebas unitarias ligeras que nos permiten detectar errores de forma rápida y mantener la calidad del código.

En esta unidad, aprenderás a:

- Usar `print` y depuradores para entender el flujo y estado de tus funciones.
- Escribir pruebas simples con `assert` para validar resultados esperados.

El Paradigma de la Calidad: ¿Por qué testear?

En el desarrollo de software y la ciencia de datos, escribir la función es solo la mitad del trabajo. La otra mitad consiste en asegurar que dicha función sea robusta. Una función robusta es aquella que se comporta de manera predecible, incluso ante entradas inesperadas.

La depuración (debugging) y el testing no son tareas "extra"; son componentes intrínsecos del flujo de trabajo. Un error en una función de limpieza de datos, por ejemplo, puede invalidar semanas de análisis estadístico si no se detecta a tiempo.

La Anatomía de la Depuración (Debugging)

Depurar es el proceso de identificar, localizar y corregir errores (bugs). Podemos clasificar los errores en tres categorías:

- **Errores de Sintaxis:** El código no se puede ejecutar (Python nos avisa de inmediato).
- **Errores de Ejecución (Runtime):** El código falla mientras corre (ej. una división por cero).
- **Errores de Lógica:** El código corre perfectamente, pero el resultado es **incorrecto**. Estos son los más peligrosos.

Herramientas de Inspección

1. **Print Debugging:** Es la técnica de insertar sentencias `print()` en puntos estratégicos para observar el "estado" de las variables en tiempo real. Aunque es rudimentaria, es extremadamente efectiva para entender el flujo en scripts lineales.
2. **Depuradores (Debuggers):** Herramientas como el depurador integrado de VS Code o el de Google Colab permiten ejecutar el código línea por línea, inspeccionar la memoria y detener la ejecución en "puntos de interrupción" (breakpoints) sin necesidad de llenar el código de prints.

Pruebas Unitarias Ligeras con assert

La programación defensiva propone que debemos "esperar lo mejor, pero estar preparados para lo peor". La instrucción assert (afirmar) es nuestra primera línea de defensa.

La Filosofía de assert

Un assert es una declaración de verdad. Si la condición que sigue al assert es verdadera, Python continúa silenciosamente. Si es falsa, el programa se detiene inmediatamente y lanza un AssertionError.

¿Por qué usarlos?

- **Documentación viva:** Un `assert` le dice a otros programadores qué resultado exacto se espera de la función.
- **Detección temprana:** Es preferible que el código falle apenas algo sale mal, antes de que el error se propague por todo el sistema (efecto cascada).

Casos de Prueba (Test Cases) y Casos Borde

Al diseñar pruebas, no basta con probar el "camino feliz" (donde todo sale bien). Un buen experto en Python siempre considera los Edge Cases (Casos Borde):

¿Qué pasa si la lista está vacía?

¿Qué pasa si el número es negativo?

¿Qué pasa si el tipo de dato es incorrecto (un string en vez de un int)?

Técnicas básicas de depuración

- **Print debugging:** Insertar `print()` en puntos clave para inspeccionar valores.
- **Depuradores:** Herramientas que permiten pausar la ejecución y examinar variables.

Pruebas unitarias simples

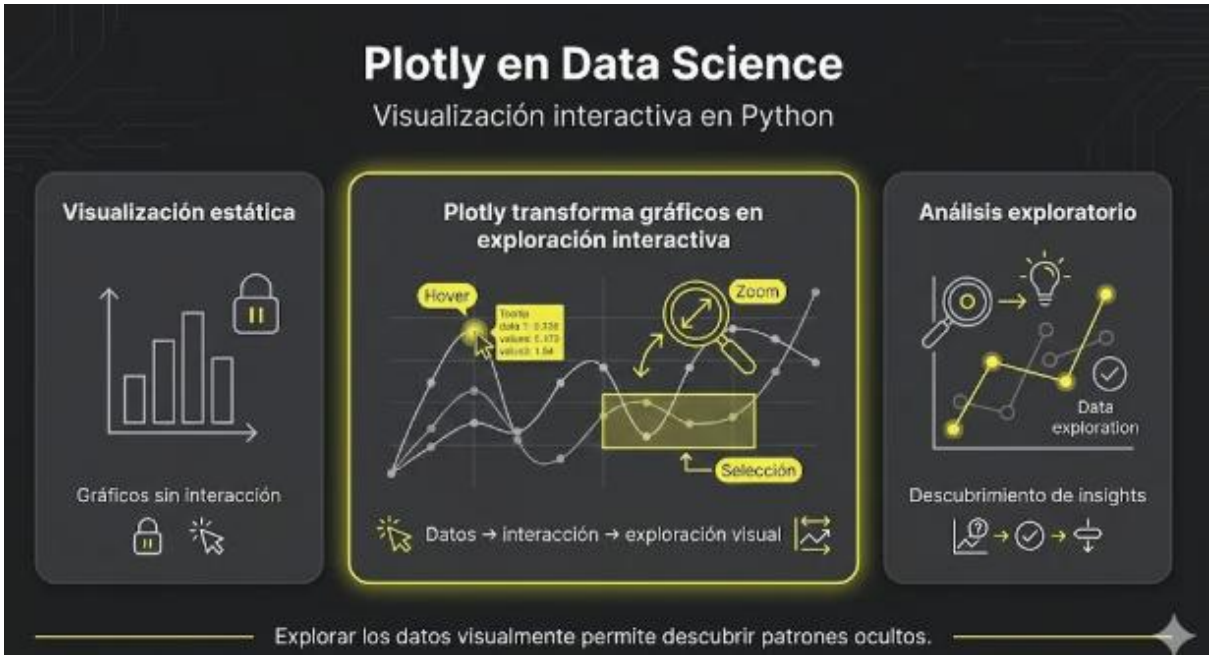
Usar `assert` para comprobar que una función devuelve el resultado esperado:

python
```
assert mi_funcion(entrada) == salida_esperada
```

Esto ayuda a detectar errores automáticamente.

---

10 - Visualización interactiva con Plotly



Qué es Plotly?

Es una librería para crear gráficos interactivos en HTML, compatibles con:

-Jupyter Notebook

-Google Colab

-Aplicaciones web (Dash)

-Exportación a HTML y PNG

□ ¿Cómo funciona?

Plotly tiene dos APIs principales:

1. plotly.express (px)

La forma más simple: crea gráficos complejos con muy pocas líneas.

Ejemplo:

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species")
fig.show()
```

Esto genera un scatter interactivo donde podés:

-hacer hover para ver valores,

-hacer zoom,

-mover el gráfico,

-guardar la imagen.

---

2. plotly.graph_objects (go)

Es la API avanzada: permite control completo sobre cada elemento del gráfico.

Ejemplo:

```
import plotly.graph_objects as go
fig = go.Figure()fig.add_trace(go.Bar(x=["A", "B", "C"], y=[10, 20,
15]))fig.update_layout(title="Gráfico de barras interactivo")fig.show()
```

□ ¿Por qué es tan usado?

✓ Interactivo por defecto

✓ Fácil para análisis exploratorio

✓ Dash permite construir dashboards completos en Python

✓ Visuales modernos

✓ Integra con Pandas y NumPy

En esta unidad aprenderás a construir visualizaciones con plotly express para gráficos rápidos y con graph_objects para mayor personalización. También veremos cómo agregar interactividad básica y exportar los gráficos a archivos HTML para compartir.

Descripción del problema

Se te proporcionará un conjunto de datos con valores numéricos y categorías. Tu tarea es generar un gráfico interactivo de barras que muestre la suma de los valores por categoría. El gráfico debe permitir interacción básica (hover para ver valores) y debe ser exportable a un archivo HTML.

Para este ejercicio, recibirás por entrada estándar una lista de pares categoría-valor. Debes procesar los datos, crear el gráfico interactivo y guardar el archivo HTML con un nombre específico.

Formato de entrada

- La primera línea contiene un entero N, el número de pares categoría-valor.

- Las siguientes N líneas contienen una categoría (cadena sin espacios) y un valor entero, separados por espacio.

  Ejemplo:

  ```
  5A 10B 20A 5C 7B 3
  ```

Formato de salida

- Debes imprimir en la salida estándar la suma total de valores por categoría, ordenadas alfabéticamente por categoría.
- Cada línea debe contener la categoría y la suma, separados por espacio.

  Ejemplo para la entrada anterior:

  ```
  A 15B 23C 7
  ```

  Además, debes generar un archivo HTML llamado `grafico.html` que contenga el gráfico interactivo de barras con las categorías en el eje X y la suma de valores en el eje Y.

Requisitos técnicos

- Usa `plotly.express` o `plotly.graph_objects` para crear el gráfico.
- El gráfico debe ser interactivo (mostrar valores al pasar el mouse).
- Exporta el gráfico a un archivo HTML llamado `grafico.html`.

Ejemplo de ejecución

Entrada:

```
4X 12Y 7X 3Z 10
```

Salida:

```
X 15Y 7Z 10
```

Se genera un archivo `grafico.html` con el gráfico interactivo correspondiente.

---

11 - Buenas prácticas: modularidad y manejo de errores

Introducción

Cuando desarrollamos funciones reutilizables, es fundamental asegurarnos de que funcionan correctamente. Para ello, usamos técnicas de depuración y pruebas unitarias ligeras que nos permiten detectar errores de forma rápida y mantener la calidad del código.

En esta unidad, aprenderás a:

-Usar print y depuradores para entender el flujo y estado de tus funciones.

-Escribir pruebas simples con assert para validar resultados esperados.

El Paradigma de la Calidad: ¿Por qué testear?

En el desarrollo de software y la ciencia de datos, escribir la función es solo la mitad del trabajo. La otra mitad consiste en asegurar que dicha función sea robusta. Una función robusta es aquella que se comporta de manera predecible, incluso ante entradas inesperadas.

La depuración (debugging) y el testing no son tareas "extra"; son componentes intrínsecos del flujo de trabajo. Un error en una función de limpieza de datos, por ejemplo, puede invalidar semanas de análisis estadístico si no se detecta a tiempo.

La Anatomía de la Depuración (Debugging)

Depurar es el proceso de identificar, localizar y corregir errores (bugs). Podemos clasificar los errores en tres categorías:

- **Errores de Sintaxis:** El código no se puede ejecutar (Python nos avisa de inmediato).
- **Errores de Ejecución (Runtime):** El código falla mientras corre (ej. una división por cero).
- **Errores de Lógica:** El código corre perfectamente, pero el resultado es **incorrecto**. Estos son los más peligrosos.

Herramientas de Inspección

1. **Print Debugging:** Es la técnica de insertar sentencias `print()` en puntos estratégicos para observar el "estado" de las variables en tiempo real. Aunque es rudimentaria, es extremadamente efectiva para entender el flujo en scripts lineales.
2. **Depuradores (Debuggers):** Herramientas como el depurador integrado de VS Code o el de Google Colab permiten ejecutar el código línea por línea, inspeccionar la memoria y detener la ejecución en "puntos de interrupción" (breakpoints) sin necesidad de llenar el código de prints.

Pruebas Unitarias Ligeras con assert

La programación defensiva propone que debemos "esperar lo mejor, pero estar preparados para lo peor". La instrucción assert (afirmar) es nuestra primera línea de defensa.

La Filosofía de assert

Un assert es una declaración de verdad. Si la condición que sigue al assert es verdadera, Python continúa silenciosamente. Si es falsa, el programa se detiene inmediatamente y lanza un AssertionError.

¿Por qué usarlos?

- **Documentación viva:** Un `assert` le dice a otros programadores qué resultado exacto se espera de la función.
- **Detección temprana:** Es preferible que el código falle apenas algo sale mal, antes de que el error se propague por todo el sistema (efecto cascada).

Casos de Prueba (Test Cases) y Casos Borde

Al diseñar pruebas, no basta con probar el "camino feliz" (donde todo sale bien). Un buen experto en Python siempre considera los Edge Cases (Casos Borde):

¿Qué pasa si la lista está vacía?

¿Qué pasa si el número es negativo?

¿Qué pasa si el tipo de dato es incorrecto (un string en vez de un int)?

Técnicas básicas de depuración

- **Print debugging:** Insertar `print()` en puntos clave para inspeccionar valores.
- **Depuradores:** Herramientas que permiten pausar la ejecución y examinar variables.

Pruebas unitarias simples

Usar assert para comprobar que una función devuelve el resultado esperado:

python
```
assert mi_funcion(entrada) == salida_esperada
```

Esto ayuda a detectar errores automáticamente.

---

12 - Fundamentos de Estadística y EDA

Estadística Descriptiva: Concepto y Relevancia

La estadística descriptiva es una rama de la estadística que se encarga de recopilar, organizar, describir y analizar un conjunto de datos con el fin de extraer información relevante. Su objetivo principal es ofrecer una visión general y comprensible de los datos, lo que permite identificar patrones, tendencias y relaciones dentro del conjunto de datos.

La importancia de la estadística descriptiva radica en su capacidad para proporcionar un resumen claro y conciso de los datos, lo cual es esencial para la toma de decisiones informadas. Al utilizar medidas de resumen como la media, mediana, moda, varianza y desviación estándar, podemos obtener una comprensión más profunda de la distribución de los datos y de las características principales del conjunto de datos analizado.

Introducción al Análisis Exploratorio de Datos (EDA)

El Análisis Exploratorio de Datos (EDA, por sus siglas en inglés) es una fase crítica en el análisis de datos que se centra en descubrir patrones, detectar anomalías, probar hipótesis y verificar supuestos mediante el uso de estadísticas descriptivas y herramientas de visualización. La filosofía detrás del EDA es acercarse a los datos sin suposiciones previas, permitiendo así una exploración abierta que puede revelar aspectos inesperados o interesantes de los datos.

El EDA no solo ayuda a entender la estructura y características de los datos, sino que también es fundamental para preparar los datos para análisis más complejos, como la modelización predictiva. Mediante el uso de gráficos, tablas y medidas de resumen, el EDA permite a los analistas de datos tomar decisiones informadas sobre la adecuación de los datos para los análisis posteriores.

El EDA no solo ayuda a entender la estructura y características de los datos, sino que también es fundamental para preparar los datos para análisis más complejos, como la modelización predictiva. Mediante el uso de gráficos, tablas y medidas de resumen, el EDA permite a los analistas de datos tomar decisiones informadas sobre la adecuación de los datos para los análisis posteriores.

Resumen

La estadística descriptiva es crucial en el análisis de datos, ya que permite resumir y comprender los datos de manera efectiva. Es una herramienta clave para el Análisis Exploratorio de Datos (EDA), que se utiliza para explorar y analizar los datos en profundidad,

ayudando a revelar patrones, anomalías y otras características importantes antes de realizar análisis más complejos.

Medidas de Tendencia Central y Dispersión

Uso de la Mediana: La mediana es especialmente útil en distribuciones asimétricas o cuando existen valores atípicos, ya que no se ve afectada por estos como la media.

Moda

La moda es el valor que aparece con mayor frecuencia en un conjunto de datos. Un conjunto de datos puede tener una única moda (unimodal), más de una moda (multimodal) o ninguna moda si todos los valores son únicos.

Uso de la Moda: La moda se utiliza principalmente en variables cualitativas o en situaciones donde se requiere conocer el valor más común dentro del conjunto de datos.

Varianza

La varianza mide la dispersión de los datos respecto a la media. Se calcula promediando el cuadrado de las diferencias entre cada valor y la media. La varianza ofrece una idea de cuán dispersos están los datos; una varianza alta indica que los datos están muy dispersos, mientras que una varianza baja indica que los datos están más concentrados alrededor de la media.

Fórmula de la varianza:

Varianza=∑i=1n(xi−Media)2n−1\text{Varianza} = \frac{\sum_{i=1}^{n} (x_i - \text{Media})^2}{n-1}

Uso de la Varianza: La varianza es fundamental para entender la variabilidad dentro de un conjunto de datos, y es una base para el cálculo de otras medidas como la desviación estándar.

Desviación Estándar

La desviación estándar es la raíz cuadrada de la varianza y proporciona una medida de dispersión en las mismas unidades que los datos originales. Es una medida más intuitiva que la varianza porque refleja directamente la magnitud de las desviaciones respecto a la media.

Fórmula de la desviación estándar:

Desviacio´n Esta´ndar=Varianza\text{Desviación Estándar} = \sqrt{\text{Varianza}}

Uso de la Desviación Estándar: La desviación estándar es ampliamente utilizada para entender la dispersión de los datos y es fundamental en muchas aplicaciones estadísticas, incluyendo la evaluación de la volatilidad en mercados financieros o la variabilidad en procesos de producción.

Cuartiles y Percentiles

Los cuartiles y percentiles son medidas que dividen un conjunto de datos ordenados en partes iguales, proporcionando información detallada sobre la distribución de los datos. Los cuartiles dividen los datos en cuatro partes iguales: el primer cuartil (Q1) marca el 25% de los datos más bajos, el segundo cuartil (Q2), que corresponde a la mediana, divide el conjunto en la mitad, y el tercer cuartil (Q3) marca el 75% de los datos más bajos. Los percentiles, por su parte, dividen los datos en 100 partes iguales, y son especialmente útiles para interpretar la posición relativa de un valor dentro de un conjunto de datos. Por ejemplo, si un valor se encuentra en el percentil 90, significa que el 90% de los valores en el conjunto son menores o iguales a él.

Uso de Cuartiles y Percentiles: Estas medidas son fundamentales para análisis en los que se requiere comprender la dispersión y distribución de los datos más allá de la media y la mediana, permitiendo identificar rangos y establecer comparaciones entre diferentes subconjuntos de datos.

Medidas Cualitativas

Medidas de Resumen para Variables Cualitativas

Las variables cualitativas, también conocidas como variables categóricas, son aquellas que describen cualidades o características y no pueden ser medidas numéricamente de manera directa. Ejemplos comunes de variables cualitativas incluyen el género, el estado civil, la categoría de un producto, entre otros. Para resumir este tipo de datos, se utilizan principalmente dos medidas: el conteo de observaciones y la moda.

Conteo de Observaciones

El conteo de observaciones es una medida básica pero poderosa para variables cualitativas. Consiste en contar la cantidad de veces que cada categoría aparece en el conjunto de datos. Este conteo permite obtener una visión clara de la distribución de las categorías dentro del conjunto de datos.

Uso del Conteo de Observaciones: Esta medida es útil para entender la frecuencia de cada categoría y para comparar cómo se distribuyen las diferentes categorías en el conjunto de datos. Por ejemplo, en un

análisis de mercado, el conteo de observaciones puede mostrar cuántos clientes pertenecen a cada segmento de mercado.

Cálculo de la Moda

La moda es la categoría que aparece con mayor frecuencia en un conjunto de datos cualitativos. A diferencia de la media y la mediana, que son más comunes en variables cuantitativas, la moda es la medida de tendencia central más adecuada para variables cualitativas. Un conjunto de datos puede ser unimodal (una sola moda), bimodal (dos modas) o multimodal (más de dos modas).

Uso de la Moda: La moda es particularmente útil cuando se quiere identificar la categoría más representativa o común en el conjunto de datos. Por ejemplo, en un estudio de preferencias de productos, la moda indicaría cuál es el producto preferido por la mayoría de los encuestados.

Distribuciones de Variables

Distribuciones Uniformes

Una distribución uniforme es aquella en la que todos los valores posibles dentro de un rango tienen la misma probabilidad de ocurrir. En otras palabras, cada resultado es igualmente probable. Esta distribución es a menudo representada por una forma rectangular en un histograma, donde la altura de cada barra es igual, reflejando que todas las categorías tienen la misma frecuencia.

Ejemplo de Distribución Uniforme: Un ejemplo clásico de distribución uniforme es el lanzamiento de un dado justo, donde cada número del 1 al 6 tiene la misma probabilidad de salir (1/6).

Distribuciones Normales

La distribución normal, también conocida como la campana de Gauss, es una de las distribuciones más importantes en estadística. Se caracteriza por ser simétrica respecto a su media y tener una forma de campana. La mayor parte de los datos se agrupa alrededor de la media, y la probabilidad disminuye a medida que nos alejamos de la media en ambos sentidos. La media, mediana y moda coinciden en el centro de la distribución.

Ejemplo de Distribución Normal: Un ejemplo típico de una distribución normal es la estatura de los adultos de una población, donde la mayoría de las personas tienen estaturas cercanas al promedio, y hay pocas personas muy bajas o muy altas.

Diferencias entre Distribuciones Uniformes y Normales

La principal diferencia entre una distribución uniforme y una normal es la forma de la distribución de los datos. Mientras que la distribución uniforme muestra que todos los resultados son igualmente probables, la distribución normal indica que los resultados cercanos a la media son más probables, y los extremos son menos probables.

- **Forma:** La distribución uniforme es rectangular, mientras que la distribución normal tiene forma de campana.

  Simetría: Aunque ambas distribuciones pueden ser simétricas, la distribución normal tiene una concentración de datos alrededor de la media, mientras que la uniforme no tiene tal concentración.

---

13 - Estadística descriptiva aplicada y Control de Versiones

¿Qué es la estadística descriptiva y por qué importa?\

La estadística descriptiva resume y organiza datos para que podamos entenderlos rápidamente: cantidad, tendencia central (dónde se concentra la información), y dispersión (qué tan dispersos están los datos). Antes de modelar o tomar decisiones, debemos *conocer* nuestros datos: ese es el objetivo del EDA (Análisis Exploratorio de Datos).

Medidas de tendencia central

- **Media**: promedio aritmético. Sensible a valores extremos (outliers).
- **Mediana**: valor central. Robusta frente a outliers.
- **Moda**: valor más frecuente. Útil en variables categóricas o cuando interesa lo más típico.

Medidas de dispersión

- **Varianza**: promedio de los cuadrados de las diferencias respecto a la media (mide variabilidad).
- **Desviación estándar**: raíz cuadrada de la varianza (misma unidad que los datos).
- **Rango intercuartílico (IQR)**: Q3 − Q1, útil para detectar outliers.

Cuartiles y percentiles

- **Q1** (25%), **Q2** (50% = mediana), **Q3** (75%).
- Un percentil p indica el valor debajo del cual se encuentra el p% de las observaciones.

Medidas para variables cualitativas

- **Conteos** (frecuencias) y **proporciones**.
- **Moda** para identificar la categoría más común.

Formas de distribución

- **Normal** (campana): media, mediana y moda coinciden; muchos métodos estadísticas asumen aproximación a normalidad.
- **Uniforme**: todos los valores en un rango tienen probabilidad similar.
- **Asimetría (skewness)** y **curtosis** nos dan pistas sobre forma y «colas» de la distribución.

---

14 - Diseño de pipelines reproducibles y mini-proyecto

Implementa un pipeline reproducible que procesa datos, realiza transformaciones y genera visualizaciones, aplicando buenas prácticas de rendimiento y modularidad.
Título del mini-proyecto: *Análisis integrador de ventas e-commerce — pipeline reproducible y código idiomático*

Esta práctica integradora propone a los estudiantes aplicar conocimientos de estilo Pythonic, manejo de tipos y colecciones, NumPy, Pandas, visualización estática e interactiva, estadística descriptiva y pruebas ligeras para construir un pequeño pipeline reproducible. El proyecto fuerza decisiones reales: limpieza, transformación, diseño de funciones, verificación con assert, y comunicación (gráficos + reporte).
Verás:

- **Python idiomático y tipos:** usan comprehensions, unpacking, context managers y cuidado con mutabilidad al construir funciones que no muten entradas.
- **NumPy & rendimiento:** generación y operaciones vectorizadas para métricas agregadas.
- **Pandas:** carga, selección, transformación, groupby y manejo de nulos.
- **Visualización (matplotlib / plotly):** comunicar hallazgos con gráficas estáticas e interactivas.
- **Estadística descriptiva y EDA:** calcular media/mediana/moda, varianza, IQR y percentiles; detectar outliers.
- **Testing ligero & depuración:** asserts y mensajes informativos para validar resultados.
- **Buenas prácticas & modularidad:** funciones puras, docstrings y guardado de artefactos (CSV, PNG).
- **Control de versiones (actividad sugerida):** se pide inicializar un repo (no obligatorio en el código) para practicar commits y rama de entrega.